AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 16.3

Task 1 – Hospital Management Database

Prompt:

"#-- Create Hospital DB with Doctors, Patients, Appointments + queries for doctor appointments, patient history, and patient count per doctor."

```sql
-- Create Hospital DB with Doctors, Patients, Appointments + quer
-- Create Doctors table
CREATE TABLE Doctors (
    DoctorID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    Specialty TEXT
);

-- Create Patients table
CREATE TABLE Patients (
    PatientID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    DateOfBirth DATE
);

-- Create Appointments table
CREATE TABLE Appointments (
    AppointmentID INTEGER PRIMARY KEY,
    DoctorID INTEGER,
    PatientID INTEGER,
    AppointmentDate TEXT,
```

```sql
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);

-- Insert data
INSERT INTO Doctors (DoctorID, FirstName, LastName, Specialty) VALUES
(1, 'John', 'Doe', 'Cardiology'),
(2, 'Jane', 'Smith', 'Neurology'),
(3, 'Emily', 'Johnson', 'Pediatrics');

INSERT INTO Patients (PatientID, FirstName, LastName, DateOfBirth) VALU
(1, 'Michael', 'Brown', '1980-05-15'),
(2, 'Sarah', 'Davis', '1990-08-22'),
(3, 'David', 'Wilson', '1975-12-30');

INSERT INTO Appointments (AppointmentID, DoctorID, PatientID, Appointme
(1, 1, 1, '2024-07-01 10:00:00'),
(2, 1, 2, '2024-07-01 11:00:00'),
(3, 2, 3, '2024-07-02 09:00:00'),
(4, 3, 1, '2024-07-03 14:00:00');

-- Query 1
SELECT a.AppointmentID, p.FirstName AS PatientFirstName, p.LastNa
FROM Appointments a
```

```sql
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = 1;

-- Query 2
SELECT a.AppointmentID, d.FirstName AS DoctorFirstName, d.LastNam
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE a.PatientID = 1;

-- Query 3
SELECT d.FirstName, d.LastName, COUNT(a.PatientID) AS PatientCoun
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID;
```

SQL 1 / 1 1 - 0 of 0

SQL 1 / 1 1 - 2 of 2

AppointmentID	PatientFirstName	PatientLastName	AppointmentDate
1	Michael	Brown	2024-07-01 10:00:00
2	Sarah	Davis	2024-07-01 11:00:00

SQL 1 / 1 1 - 2 of 2

AppointmentID	DoctorFirstName	DoctorLastName
1	John	Doe
4	Emily	Johnson

SQL 1 / 1 1 - 3 of 3

FirstName	LastName	PatientCount
John	Doe	2
Jane	Smith	1
Emily	Johnson	1

Explanation:

- AI was used to generate a normalized Hospital database with Doctors, Patients, and Appointments tables.
- Relationships were established using primary and foreign keys to maintain data integrity.
- SQL queries were generated to retrieve doctor appointments, patient history, and patient count per doctor.
- The queries were executed successfully after minor SQLite syntax corrections.

Task 2 – Real-Time Application: Online Shopping Database

Prompt:“#-- Create E-commerce database (SQLite) with Users, Products, Orders, OrderDetails + queries for user orders, most purchased product, and total revenue.”

▷Run on active connection | ≡ Select block

```sql
-- Create E-commerce database (SQLite) with Users, Products, Orders, OrderDetails + queries
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;
-- Create Users table
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    Email TEXT
);
-- Create Products table
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY,
    ProductName TEXT,
    Price REAL
);
-- Create Orders table
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY,
    UserID INTEGER,
    OrderDate TEXT,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
-- Create OrderDetails table
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
-- Insert data
INSERT INTO Users (UserID, FirstName, LastName, Email) VALUES
(1, 'Alice', 'Johnson', 'alice.johnson@example.com');

INSERT INTO Products (ProductID, ProductName, Price) VALUES
(1, 'Laptop', 999.99),
(2, 'Mouse', 29.99),
(3, 'Keyboard', 79.99);

INSERT INTO Orders (OrderID, UserID, OrderDate) VALUES
(1, 1, '2024-07-01 10:00:00');

INSERT INTO OrderDetails (OrderDetailID, OrderID, ProductID, Quantity) VALUES
```

```sql
INSERT INTO OrderDetails (OrderDetailID, OrderID, ProductID, Quantity) VALUES
(1, 1, 1, 1),
(2, 1, 2, 2);
-- Query 1: User orders
SELECT o.OrderID, p.ProductName, od.Quantity, (p.Price * od.Quantity) AS TotalPrice
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE o.UserID = 1;
-- Query 2: Most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductName
ORDER BY TotalQuantity DESC
LIMIT 1;
-- Query 3: Total revenue
SELECT SUM(p.Price * od.Quantity) AS TotalRevenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID;
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 2 of 2

OrderID	ProductName	Quantity	TotalPrice
1	Laptop	1	999.99
1	Mouse	2	59.98

SQL ▼ < 1 / 1 > 1 - 1 of 1

ProductName	TotalQuantity
Mouse	2

SQL ▼ < 1 / 1 > 1 - 1 of 1

TotalRevenue
1059.97

Explanation:

- AI generated an E-commerce database with Users, Products, Orders, and OrderDetails tables.
- Relationships were defined using foreign keys to connect users, orders, and products.

- Queries were used to retrieve user orders, identify the most purchased product, and calculate total revenue.
- The queries executed successfully and provided correct analytical results.

Task 3 – Library Database

Prompt:

“#-- Create Library DB (SQLite) with Books, Members, Loans + queries for issued books, overdue books, and books count per member.”

```
▷Run on active connection | ☰Select block
-- Create Library DB (SQLite) with Books, Members, Loans + queries for issued books, overdue
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;
-- Create Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY,
    Title TEXT,
    Author TEXT,
    PublishedYear INTEGER
);
-- Create Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    MembershipDate DATE
);
-- Create Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY,
    BookID INTEGER,
    MemberID INTEGER,
```

```sql
    LoanDate TEXT,
    ReturnDate TEXT,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Insert data
INSERT INTO Books (BookID, Title, Author, PublishedYear) VALUES
(1, 'To Kill a Mockingbird', 'Harper Lee', 1960),
(2, '1984', 'George Orwell', 1949),
(3, 'The Great Gatsby', 'F. Scott Fitzgerald', 1925);
INSERT INTO Members (MemberID, FirstName, LastName, MembershipDate) VALUES
(1, 'Alice', 'Johnson', '2020-01-15'),
(2, 'Bob', 'Smith', '2021-03-22'),
(3, 'Charlie', 'Brown', '2019-07-30');
INSERT INTO Loans (LoanID, BookID, MemberID, LoanDate, ReturnDate) VALUES
(1, 1, 1, '2024-07-01', '2024-07-10'),
(2, 2, 1, '2024-07-05', NULL),
(3, 3, 2, '2024-07-03', '2024-07-12');
-- Query 1: Issued books
SELECT l.LoanID, b.Title, m.FirstName || ' ' || m.LastName AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
```

```sql
WHERE l.ReturnDate IS NULL;
-- Query 2: Overdue books (assuming current date is '2024-07-15')
SELECT l.LoanID, b.Title, m.FirstName || ' ' || m.LastName AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < '2024-07-01';
-- Query 3: Books count per member
SELECT m.FirstName || ' ' || m.LastName AS MemberName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 1 of 1

LoanID	Title	MemberName	LoanDate
2	1984	Alice Johnson	2024-07-05

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 3 of 3

MemberName	BooksLoaned
Alice Johnson	2
Bob Smith	1
Charlie Brown	0

Explanation:

- AI generated a Library database with Books, Members, and Loans tables.
- Relationships were established using foreign keys between books and members.

- Queries were used to retrieve issued books, find overdue books, and count books per member.
- The queries executed successfully and returned correct results.

Task 4: Optimize Queries

Prompt:

"#-- Optimize SQL query by replacing subquery with JOIN and compare performance."

```sql
-- Create Authors table
CREATE TABLE Authors (
    author_id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);

-- Create Books table
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)
);

-- Insert Authors
INSERT INTO Authors VALUES
(1, 'Author A', 'UK'),
(2, 'Author B', 'USA'),
(3, 'Author C', 'UK');

-- Insert Books
INSERT INTO Books VALUES
(1, 'Book1', 1),
```

```sql
(2, 'Book2', 2),
(3, 'Book3', 3);

-- ORIGINAL QUERY (Subquery)
SELECT * FROM Books
WHERE author_id IN (
    SELECT author_id FROM Authors WHERE country = 'UK'
);

-- OPTIMIZED QUERY (JOIN)
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 2 of 2

book_id	title	author_id
1	Book1	1
3	Book3	3

SQL ▼ 1 / 1 1 - 2 of 2

book_id	title	author_id
1	Book1	1
3	Book3	3

Explanation:

- The original query uses a **subquery** to filter books by author country, which requires executing an inner query first and can be slower for large datasets.
- The optimized query replaces the subquery with a **JOIN**, directly combining Books and Authors tables, making execution more efficient.

- JOIN reduces repeated scanning and improves performance by handling data in a single step.
- Both queries return the **same result**, but the optimized query is faster and more scalable.

Task 5: University Course Registration

Prompt:

"#-- Create University DB (SQLite) with Students, Courses, Faculty, Registrations + queries for students in a course, faculty teaching >2 courses, and most registered course."

```sql
-- Create University DB (SQLite) with Students, Courses, Faculty, Registrations + queries fo
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;
-- Create Students table
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    EnrollmentDate DATE
);
-- Create Courses table
CREATE TABLE Courses (
    CourseID INTEGER PRIMARY KEY,
    CourseName TEXT,
    FacultyID INTEGER,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);
-- Create Faculty table
CREATE TABLE Faculty (
    FacultyID INTEGER PRIMARY KEY,
    FirstName TEXT,
```

```sql
    LastName TEXT,
    Department TEXT
);
-- Create Registrations table
CREATE TABLE Registrations (
    RegistrationID INTEGER PRIMARY KEY,
    StudentID INTEGER,
    CourseID INTEGER,
    RegistrationDate TEXT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
-- Insert data
INSERT INTO Students (StudentID, FirstName, LastName, EnrollmentDate) VALUES
(1, 'Alice', 'Johnson', '2020-01-15'),
(2, 'Bob', 'Smith', '2021-03-22'),
(3, 'Charlie', 'Brown', '2019-07-30');
INSERT INTO Faculty (FacultyID, FirstName, LastName, Department) VALUES
(1, 'Dr. John', 'Doe', 'Computer Science'),
(2, 'Dr. Jane', 'Smith', 'Mathematics'),
(3, 'Dr. Emily', 'Johnson', 'Physics');
INSERT INTO Courses (CourseID, CourseName, FacultyID) VALUES
(1, 'Intro to Computer Science', 1),
(2, 'Calculus I', 2),
```

```sql
(3, 'Physics I', 3),
(4, 'Data Structures', 1);
INSERT INTO Registrations (RegistrationID, StudentID, CourseID, RegistrationDate) VALUES
(1, 1, 1, '2020-09-01'),
(2, 1, 2, '2020-09-01'),
(3, 2, 1, '2021-09-01'),
(4, 2, 3, '2021-09-01'),
(5, 3, 2, '2019-09-01'),
(6, 3, 4, '2019-09-01');
-- Query 1: Students in a course
SELECT s.FirstName, s.LastName
FROM Students s
JOIN Registrations r ON s.StudentID = r.StudentID
WHERE r.CourseID = 1;
-- Query 2: Faculty teaching >2 courses
SELECT f.FirstName, f.LastName
FROM Faculty f
JOIN Courses c ON f.FacultyID = c.FacultyID
GROUP BY f.FacultyID
HAVING COUNT(c.CourseID) > 2;
-- Query 3: Most registered course
SELECT c.CourseName, COUNT(r.StudentID) AS StudentCount
FROM Courses c
JOIN Registrations r ON c.CourseID = r.CourseID
```

```sql
JOIN Registrations r ON c.CourseID = r.CourseID
GROUP BY c.CourseID
ORDER BY StudentCount DESC
LIMIT 1;
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 2 of 2

FirstName	LastName
Alice	Johnson
Bob	Smith

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 1 of 1

CourseName	StudentCount
Intro to Computer Science	2

Explanation:

- AI generated a University database with Students, Courses, Faculty, and Registrations tables.
- Relationships were defined to allow students to enroll in multiple courses and link courses to faculty.
- Queries were used to retrieve students in a course, identify faculty handling multiple courses, and find the most popular course.
- The queries executed successfully and produced correct results.

Conclusion:

AI-assisted coding significantly simplified the process of designing database schemas and generating SQL queries for various real-world applications. It enabled faster development and helped in identifying and fixing errors efficiently, including query optimization. Overall, the lab demonstrated how AI tools can improve productivity, accuracy, and understanding in database development.